Faculdade de Computação e Informática – FCI
Laboratório de Organização de Computadores
Prof. Dr. Jean M. Laine

Relatório de Atividade Prática

NÚMERO DO LABORATÓRIO #3

Integrantes do Grupo (EM ORDEM ALFABÉTICA)

Nome: Dandi Mateus Aguiar Reis RA: 10444938
Nome: Leandro Solovjovas dos Santos RA: 10438426
Nome: Pedro Henrique Marques Ferreira RA: 10723681

EXERCÍCIO 1

PRINT DO CÓDIGO FONTE

```
# ============================================================
# Laboratorio 3 - Exercicio 1
# Bubble Sort com Alocacao Dinamica e Contadores
#
# Dandi Mateus Aguiar Reis              RA: 10444938
# Leandro Solovjovas dos Santos         RA: 10438426
# Pedro Henrique Marques Ferreira       RA: 10723681
# ============================================================

.data

msgN: .asciiz "\nDigite a quantidade de numeros: "
msgErroN: .asciiz "\nErro! Deve ser maior que zero.\n"
msgNumero: .asciiz "Digite um numero: "
msgVetor: .asciiz "\nVetor ordenado: "
msgComparacoes: .asciiz "\nTotal de comparacoes: "
msgTrocas: .asciiz "\nTotal de trocas: "
separador: .asciiz " "
novaLinha: .asciiz "\n"

.text
.globl main

#############################################################
# Programa Principal
#############################################################

main:

#############################################################
# Leitura e validacao da quantidade de elementos
#############################################################

lerN:

    li $v0, 4
    la $a0, msgN
    syscall

    li $v0, 5
    syscall
    move $s1, $v0

    # Garante que N seja maior que zero
    bgtz $s1, alocarVetor

    li $v0, 4
    la $a0, msgErroN
    syscall

    j lerN
```

```
53  ############################################################
54  # Alocacao dinamica do vetor na heap
55  ############################################################
56
57  alocarVetor:
58
59      # Cada inteiro ocupa 4 bytes: tamanho = N * 4
60      sll $a0, $s1, 2
61      li $v0, 9
62      syscall
63      move $s0, $v0
64
65  ############################################################
66  # Leitura e armazenamento dos elementos
67  ############################################################
68
69      li $t0, 0
70
71  lerElementos:
72
73      bge $t0, $s1, ordenarVetor
74
75      li $v0, 4
76      la $a0, msgNumero
77      syscall
78
79      li $v0, 5
80      syscall
81
82      # Calcula o endereco do elemento: base + indice * 4
83      sll $t1, $t0, 2
84      addu $t2, $s0, $t1
85      sw $v0, 0($t2)
86
87      addi $t0, $t0, 1
88      j lerElementos
89
90  ############################################################
91  # Chamada da rotina Bubble Sort
92  ############################################################
93
94  ordenarVetor:
95
96      move $a0, $s0
97      move $a1, $s1
98      jal bubbleSort
99
100     # A rotina retorna comparacoes em $v0 e trocas em $v1
101     move $s2, $v0
102     move $s3, $v1
103
104 ############################################################
105 # Impressao do vetor ordenado
106 ############################################################
107
108     li $v0, 4
109     la $a0, msgVetor
110     syscall
111
112     li $t0, 0
113
114 imprimirVetor:
115
116     bge $t0, $s1, imprimirContadores
117
118     sll $t1, $t0, 2
119     addu $t2, $s0, $t1
120     lw $a0, 0($t2)
121
```

```
    li $v0, 1
    syscall

    li $v0, 4
    la $a0, separador
    syscall

    addi $t0, $t0, 1
    j imprimirVetor

############################################################
# Impressao das comparacoes e trocas
############################################################

imprimirContadores:

    li $v0, 4
    la $a0, msgComparacoes
    syscall

    li $v0, 1
    move $a0, $s2
    syscall

    li $v0, 4
    la $a0, msgTrocas
    syscall

    li $v0, 1
    move $a0, $s3
    syscall

    li $v0, 4
    la $a0, novaLinha
    syscall

############################################################
# Encerramento do programa
############################################################

    li $v0, 10
    syscall

############################################################
# Rotina Bubble Sort
# Entrada: $a0 = endereco do vetor, $a1 = quantidade
# Saida: $v0 = comparacoes, $v1 = trocas
############################################################

bubbleSort:

    move $t6, $a0
    addi $t0, $a1, -1
    li $v0, 0
    li $v1, 0

    # Vetores com zero ou um elemento nao precisam de trocas
    blez $t0, fimBubble

loopExterno:

    li $t1, 0

loopInterno:

    bge $t1, $t0, fimPassagem

    # Calcula o endereco do elemento atual
    sll $t2, $t1, 2
    addu $t3, $t6, $t2
```

```
192
193     lw $t4, 0($t3)
194     lw $t5, 4($t3)
195
196     # Cada comparacao entre elementos adjacentes e contabilizada
197     addi $v0, $v0, 1
198
199     # Troca os elementos se estiverem fora de ordem
200     ble $t4, $t5, proximoElemento
201
202     sw $t5, 0($t3)
203     sw $t4, 4($t3)
204     addi $v1, $v1, 1
205
206 proximoElemento:
207
208     addi $t1, $t1, 1
209     j loopInterno
210
211 fimPassagem:
212
213     addi $t0, $t0, -1
214     bgtz $t0, loopExterno
215
216 fimBubble:
217
218     jr $ra
```

PRINT COM TESTES DE FUNCIONALIDADE

```
Digite a quantidade de numeros: 5
Digite um numero: 5
Digite um numero: 2
Digite um numero: 4
Digite um numero: 1
Digite um numero: 3

Vetor ordenado: 1 2 3 4 5
Total de comparacoes: 10
Total de trocas: 7

-- program is finished running --
```

```
Digite a quantidade de numeros: 4
Digite um numero: 1
Digite um numero: 2
Digite um numero: 3
Digite um numero: 4

Vetor ordenado: 1 2 3 4
Total de comparacoes: 6
Total de trocas: 0

-- program is finished running --
```

```
Digite a quantidade de numeros: -1

Erro! Deve ser maior que zero.

Digite a quantidade de numeros: 2
Digite um numero: 3
Digite um numero: 1

Vetor ordenado: 1 3
Total de comparacoes: 1
Total de trocas: 1
```

COMENTÁRIOS GERAIS

A solução foi desenvolvida em Assembly MIPS utilizando alocação dinâmica de memória para armazenar os elementos de um vetor de inteiros. A lógica consiste em receber a quantidade de elementos, validar a entrada, ler os valores informados e utilizar o algoritmo BubbleSort para ordená-los em ordem crescente. Durante a ordenação, são contabilizadas as comparações e as trocas realizadas, permitindo analisar o funcionamento do algoritmo. Uma das principais dificuldades foi implementar corretamente os laços de repetição, controlar os índices do vetor e realizar a contagem das operações em Assembly. A solução atende ao que foi solicitado, contemplando a alocação dinâmica, a ordenação, a exibição do vetor ordenado e a apresentação das quantidades de comparações e trocas. Também é importante testar entradas inválidas, como uma quantidade menor ou igual a zero, para verificar se a validação funciona corretamente. Como comentário geral, o exercício contribuiu para compreender melhor a manipulação de vetores, o uso de memória dinâmica e a análise do funcionamento de um algoritmo de ordenação.

EXERCÍCIO 2

PRINT DO CÓDIGO FONTE

```
# ============================================================
# Laboratorio 3 - Exercicio 2
# Categorizacao de Numeros em Listas Encadeadas
#
# Dandi Mateus Aguiar Reis              RA: 10444938
# Leandro Solovjovas dos Santos         RA: 10438426
# Pedro Henrique Marques Ferreira       RA: 10723681
# ============================================================

.data

# Vetor para armazenar os dez numeros digitados
vetor: .space 40

# Ponteiros para o inicio das listas
headPar:   .word 0
headImpar: .word 0

msgEntrada: .asciiz "Digite 10 numeros inteiros:\n"
msgPares:   .asciiz "\nNumeros pares: "
msgImpares: .asciiz "\nNumeros impares: "
msgVazia:   .asciiz "(lista vazia)"
separador:  .asciiz " "
novaLinha:  .asciiz "\n"

.text
.globl main

# ============================================================
# Programa principal
# ============================================================

main:
    # Solicita os dez numeros ao usuario
    li $v0, 4
    la $a0, msgEntrada
    syscall

    la $s0, vetor
    li $s1, 10
    li $s2, 0

lerVetor:
    bge $s2, $s1, classificarVetor

    # Le um numero inteiro pelo console
    li $v0, 5
    syscall

    # Calcula o endereco do elemento: base + indice * 4
    sll $t0, $s2, 2
    addu $t1, $s0, $t0
    sw $v0, 0($t1)

    addi $s2, $s2, 1
    j lerVetor

```

```
# ============================================================
# Classifica os numeros em pares e impares
# ============================================================

classificarVetor:
    li $s2, 0

percorrerVetor:
    bge $s2, $s1, mostrarListas

    # Recupera o elemento atual do vetor
    sll $t0, $s2, 2
    addu $t1, $s0, $t0
    lw $a0, 0($t1)

    # O bit menos significativo determina a paridade
    andi $t2, $a0, 1
    bnez $t2, encaminharImpar

encaminharPar:
    la $a1, headPar
    jal inserirFinal
    j proximoNumero

encaminharImpar:
    la $a1, headImpar
    jal inserirFinal

proximoNumero:
    addi $s2, $s2, 1
    j percorrerVetor

```

```
# ============================================================
# Exibe as duas listas
# ============================================================

mostrarListas:
    li $v0, 4
    la $a0, msgPares
    syscall

    lw $a0, headPar
    jal imprimirLista

    li $v0, 4
    la $a0, msgImpares
    syscall

    lw $a0, headImpar
    jal imprimirLista

    li $v0, 4
    la $a0, novaLinha
    syscall

    # Encerra a execucao
    li $v0, 10
    syscall

```

```
# ============================================================
# inserirFinal
# Entrada: $a0 = valor; $a1 = endereco do head
# Cada no possui 8 bytes: valor e ponteiro next
# ============================================================

inserirFinal:
    # Salva os argumentos antes da syscall
    move $t7, $a0
    move $t6, $a1

    # Solicita oito bytes na heap
    li $a0, 8
    li $v0, 9
    syscall

    move $t0, $v0

    # Inicializa os campos do novo no
    sw $t7, 0($t0)
    sw $zero, 4($t0)

    # Verifica se a lista esta vazia
    lw $t1, 0($t6)
    beqz $t1, inserirComoPrimeiro

# Percorre a lista ate o ultimo no
percorrerLista:
    lw $t2, 4($t1)
    beqz $t2, conectarNo

    move $t1, $t2
    j percorrerLista

conectarNo:
    sw $t0, 4($t1)
    jr $ra

# Atualiza o head da lista vazia
inserirComoPrimeiro:
    sw $t0, 0($t6)
    jr $ra

# ============================================================
# imprimirLista
# Entrada: $a0 = endereco do primeiro no
# ============================================================

imprimirLista:
    move $t0, $a0

    # Lista vazia quando o endereco inicial e zero
    beqz $t0, imprimirListaVazia

percorrerParaImprimir:
    # Imprime o valor do no atual
    lw $a0, 0($t0)
    li $v0, 1
    syscall

    # Imprime um espaco entre os numeros
    li $v0, 4
    la $a0, separador
    syscall

    # Avanca para o proximo no
    lw $t0, 4($t0)
    bnez $t0, percorrerParaImprimir

    jr $ra

imprimirListaVazia:
    li $v0, 4
    la $a0, msgVazia
    syscall

    jr $ra
```

PRINT COM TESTES DE FUNCIONALIDADE

```
Digite 10 numeros inteiros:
2
4
6
8
10
12
14
16
18
20

Numeros pares: 2 4 6 8 10 12 14 16 18 20
Numeros impares: (lista vazia)

-- program is finished running --
```

```
Digite 10 numeros inteiros:
1
3
5
7
9
11
13
15
17
19

Numeros pares: (lista vazia)
Numeros impares: 1 3 5 7 9 11 13 15 17 19

-- program is finished running --
```

```
Digite 10 numeros inteiros:
5
8
1
40
37
29
10
3
5
11

Numeros pares: 8 40 10
Numeros impares: 5 1 37 29 3 5 11

-- program is finished running --
```

COMENTÁRIOS GERAIS

A solução foi desenvolvida em Assembly MIPS utilizando um vetor estático com dez números inteiros e duas listas encadeadas, uma para armazenar os números pares e outra para os ímpares. A lógica consiste em percorrer o vetor, verificar a paridade de cada número por meio de uma operação lógica e inserir o elemento no final da lista correspondente. Para isso, cada nó é alocado dinamicamente e contém o valor armazenado e um ponteiro para o próximo nó. Entre as principais dificuldades estão o gerenciamento dos ponteiros, a alocação de memória para cada nó, a inserção dos elementos no final das listas e o tratamento de listas vazias. A solução contempla os requisitos principais do exercício, incluindo a inicialização dos ponteiros das listas, a separação dos valores, a inserção dinâmica dos nós e a impressão dos resultados. Para avaliar seu funcionamento, é importante testar situações em que os números estejam misturados, sejam todos pares ou sejam todos ímpares. Como comentário geral, o exercício permitiu aprofundar os conhecimentos sobre estruturas de dados dinâmicas, manipulação de endereços de memória e organização de elementos por meio de listas encadeadas.

OBSERVAÇÕES FINAIS

- **Gerar um PDF** deste arquivo antes de submeter na tarefa do Moodle;
- Além deste relatório, **enviar o arquivo .asm** do(s) programa(s) criado(s): Ex1.asm, Ex2.asm ...
- Cuidado com o tamanho e qualidade das imagens. Observe se é possível visualizar seu conteúdo!
- **ATENÇÃO**: Na tarefa do Moodle, apenas 1 (um) aluno da equipe deve submeter a atividade.
- **O que será avaliado**?
    - Completude, clareza e organização do relatório. É obrigatório usar este *template*;
    - Funcionamento correto dos programas, atendendo ao que foi solicitado. O quão fiel cada programa é quanto à descrição do enunciado;
    - Completude dos testes realizados;
    - Indentação, comentários, organização e legibilidade dos códigos;